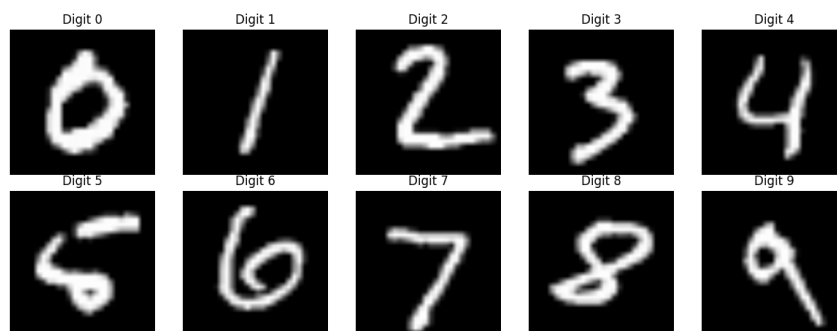


1. Zbiory danych

1.1. Charakterystyka zbioru MNIST

Pierwszym obiektem, na którym przeprowadzono badania jest popularny w uczeniu maszynowych zbiór danych MNIST (ang. *Modified National Institute of Standards and Technology*). Składający się łącznie około 70 000 obrazów (60 000 przykładów testowych oraz około 10 000 przykładów treningowych), przedstawia ręcznie pisane cyfry od 0 do 9. Każdy z obrazów domyślnie posiada rozmiar 28x28 pikseli, co w efekcie daje 784 piksele (cechy) dla każdego rozważanego obrazu. Cały zbiór danych reprezentowany jest w skali szarości, co oznacza, że każdy piksel przyjmuje wartość z zakresu od 0 do 255, gdzie 0 oznacza kolor czarny, natomiast 255 kolor biały. Przykładowe obrazy z tego zbioru danych przedstawiono na rysunku 1.1.



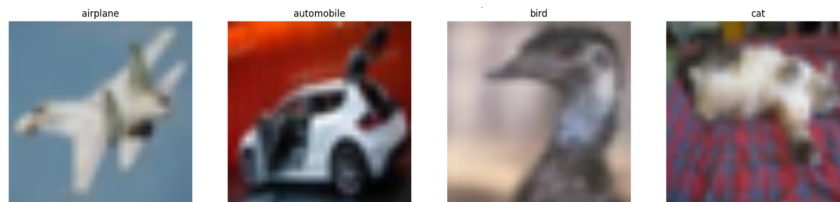
Rysunek 1.1. Przykładowe obrazy z zbioru MNIST

Cechą zbioru MNIST jest jego zbalansowany charakter, co oznacza, że każda z etykiet jest reprezentowana przez podobną ilość przykładów uczących w zbiorze. Zbalansowany zbiór danych jest kluczowy w zadaniach klasyfikacji, ponieważ pozwala na uniknięcie problemu tendencyjności modelu podczas późniejszej klasyfikacji nowych danych, których model wcześniej nie widział. Zdecydowano się na wykorzystanie opisywanego zbioru ze względu na jego popularność i prostotę. Skala szarości, która przekłada się na prostotę zbioru danych pozwala na wstępne zapoznanie się ze specyfiką wyników generowanych przez badane algorytmy wyjaśnialnej sztucznej inteligencji, co stanowi dobrą bazę do dalszych eksperymentów z bardziej złożonymi zbiorami danych.

1.2. Charakterystyka zbioru CIFAR-10

Kolejnym zbiorem danych, który wykorzystano w badaniach jest również popularny zbiór CIFAR-10 (ang. *Canadian Institute for Advanced Research*). Zbiór ten charakteryzuje się podobną ilością przykładów, co omawiany wcześniej MNIST. Podobnie jak poprzednik, CIFAR-10 posiada 10 klas, jednak w przeciwieństwie do MNIST, klasy te reprezentują różne

obiekty, takie jak samochody, samoloty, ptaki etc. Od poprzednika różni się również rozmiarem obrazów. W tym przypadku mamy do czynienia z danymi o rozmiarze 32x32 pikseli, co daje 1024 cechy. Z racji, że CIFAR-10 jest zbiorem danych kolorowych, liczbę tę należy pomnożyć przez 3 (kanały RGB), co daje łącznie 3072 cechy dla każdego obrazu. Przykładowe obrazy z tego zbioru danych przedstawiono na rysunku 1.2.



Rysunek 1.2. Przykładowe obrazy z zbioru CIFAR-10

Analogicznie jak w przypadku MNIST, CIFAR-10 jest zbiorem danych zbalansowanym, co oznacza, że każda z klas jest reprezentowana przez podobną ilość przykładów uczących. Wybór omawianego zbioru danych był podyktowany potrzebą zbadania algorytmów wyjaśnialnej sztucznej inteligencji w warunkach danych bardziej złożonych, które odzwierciedlają realistyczne scenariusze (różne gatunki zwierząt, obiekty, tło itp.). Ze względu na swoją różnorodność stanowi doskonały test dla badanych algorytmów, pozwalając na ocenę ich skuteczności w kontekście danych odzwierciedlających świat rzeczywisty.

2. Środowisko badawcze

W niniejszym rozdziale przedstawiono wszystkie elementy środowiska badawczego, które zostały wykorzystane w przeprowadzonych eksperymentach. Opisano przygotowanie konfiguracji, danych, modeli klasyfikacyjnych, metod wyjaśniania decyzji modeli, metod generowania perturbacji oraz metryki oceny wyjaśnień. W całym eksperymencie zdecydowano się na podejście obiektowe i modułowe, co oznacza, że każdy z elementów środowiska badawczego został przygotowany w sposób niezależny, a następnie połączony w jedną całość. Takie podejście pozwoliło na elastyczność i możliwość łatwej modyfikacji poszczególnych elementów w przypadku potrzeby modyfikacji parametrów eksperymentu lub wprowadzenia nowych metod wyjaśniania decyzji modeli. W kolejnych sekcjach przedstawiono szczegółowe informacje dotyczące każdego z elementów środowiska badawczego, co pozwoli na pełne zrozumienie procesu przeprowadzonych eksperymentów oraz interpretację uzyskanych wyników.

2.1. Przygotowanie konfiguracji, ścieżek oraz funkcji pomocniczych

Pracę nad środowiskiem badawczym rozpoczęto od zaimplementowania zestawu klas konfiguracyjnych oraz zestawu funkcji pomocniczych, których zadaniem było uporządkowanie parametrów eksperymentów i ograniczenie powtarzania tych samych fragmentów kodu w różnych częściach programu.

Do przechowywania parametrów wykorzystano klasy danych oznaczone dekoratorem `@dataclass`. Klasa `IGConfig` zawierała ustawienia metody Integrated Gradients, takie jak liczba kroków całkowania `n_steps` oraz wartość bazowa `baseline_value`. Wartość bazowa była wykorzystywana podczas tworzenia tensora odniesienia dla analizowanego obrazu. Klasa `OcclusionConfig` przechowywała parametry metody Occlusion. Zawierała osobne ustawienia dla zbiorów MNIST oraz CIFAR-10, obejmujące krok przesuwania okna `strides` oraz rozmiar okna okluzji `sliding_window_shapes`. Rozdzielenie tych parametrów pozwalało dostosować działanie metody do charakterystyki analizowanego zbioru danych. W klasie tej zdefiniowano również wartość bazową wykorzystywaną podczas zastępowania fragmentów obrazu.

Klasa `ExperimentConfig` grupowała parametry związane z przebiegiem eksperymentów. Obejmowała wartości odchylenia standardowego szumu Gaussa, wartości parametru `epsilon` dla ataku FGSM, procent pikseli uwzględnianych w metryce Top-K IoU, liczbę kroków w procedurze Deletion AUC oraz wartość bazową stosowaną podczas usuwania pikseli. Dzięki temu wszystkie najważniejsze parametry eksperymentalne były zebrane w jednym miejscu i mogły być łatwo modyfikowane bez ingerencji w kod poszczególnych klas. Klasa `TrainingConfig` zawierała podstawowe ustawienia procesu uczenia modelu, takie jak rozmiar batcha, liczba epok, współczynnik uczenia oraz udział zbioru walidacyjnego. Parametry te były

wykorzystywane podczas przygotowywania danych oraz trenowania modeli klasyfikacyjnych. W projekcie zdefiniowano również klasę `Paths`, której zadaniem było przechowywanie ścieżek do najważniejszych katalogów projektu. Na podstawie lokalizacji pliku wyznaczano katalog główny projektu, a następnie katalog danych oraz katalog przeznaczony do zapisu modeli.

Dodatkowe funkcje pomocnicze umieszczono w pliku `utils.py`. Funkcja `get_device()` automatycznie wybierała urządzenie obliczeniowe, zwracając GPU w przypadku dostępności technologii CUDA albo CPU w przeciwnym przypadku. Funkcja `tensor_to_img()` służyła do przekształcania tensora obrazu do postaci możliwej do wyświetlenia. W ramach tej operacji tensor był przenoszony na CPU, konwertowany do tablicy NumPy, a następnie przekształcany z zakresu wartości $[-1,1]$ do zakresu $[0,1]$. Funkcja `plot_image()` ujednolicała sposób wyświetlania obrazów i map atrybucji. Przyjmowała obiekt osi wykresu, obraz, tytuł oraz opcjonalne parametry mapy kolorów i zakresu wartości. Dzięki temu możliwe było stosowanie spójnego sposobu wizualizacji w różnych częściach eksperymentu. W pliku pomocniczym zdefiniowano także funkcję `initialize_resnet18()`, która tworzyła model ResNet18. W zależności od przekazanego parametru model mógł zostać zainicjalizowany z wagami wstępnie wytrenowanymi lub bez nich. Ostatnia warstwa była zastępowana nową warstwą, której liczba wyjść odpowiadała liczbie klas analizowanego zbioru danych. Funkcja `load_model_weights()` umożliwiała wczytanie zapisanych wag modelu, przeniesienie modelu na odpowiednie urządzenie oraz przełączenie go w tryb ewaluacji.

2.2. Przygotowanie danych oraz procesu trenowania

Następnie przystąpiono do przygotowania klasy `DataManager`, która służyła do ujednolicenia procesu przygotowania danych wykorzystywanych w badaniach. Jej głównym zadaniem było zapewnienie wspólnego interfejsu do obsługi zbioru MNIST oraz CIFAR-10, obejmującego pobieranie danych, ich wstępne przetwarzanie, wyodrębnienie ze zbioru treningowego zbioru walidacyjnego, pobranie zbioru testowego oraz obiektów `Data Loader`, które umożliwiały iterację po danych w trakcie trenowania modeli. Dzięki temu możliwe było łatwe zarządzanie danymi oraz zapewnienie spójności w procesie przygotowania danych dla eksperymentów. W konstruktorze klasy określano ścieżkę do katalogu danych na podstawie klasy `Paths` oraz tworzono obiekt `TrainingConfig`, z którego pobierano parametry związane z treningiem, takie jak rozmiar batcha oraz udział zbioru walidacyjnego. Klasa przechowywała również mapowanie indeksów klas na ich nazwy. Dla zbioru MNIST indeksy odpowiadały cyfrom od 0 do 9, natomiast dla zbioru CIFAR-10 przypisano im nazwy klas, takie jak `airplane`, `automobile`, `bird` czy `cat`. Metoda `get_class_names()` umożliwiała pobranie tego mapowania dla wskazanego zbioru danych.

Istotnym elementem klasy `DataManager` jest metoda `get_transform()`, która definiuje sposób przetwarzania obrazów w zależności od zbioru danych. Obrazy są przekształcane do rozmiaru 224x224 pikseli, następnie zamieniane na obiekty typu `tensor`. Ostatnim

etapem przygotowywania danych dla modelu jest normalizacja. W badaniach została ona przeprowadzona zgodnie ze wzorem 2.1.

$$x_{norm} = \frac{x - \mu}{\sigma} \quad (2.1)$$

gdzie:

x – wartość wejściowa piksela,

μ – średnia wartość pikseli dla danego zbioru,

σ – odchylenie standardowe wartości pikseli.

Wartości μ i σ zostały przyjęte na poziomie 0.5. Podstawiając te wartości do wzoru 2.1 otrzymujemy:

$$x_{norm} = \frac{x - 0.5}{0.5} = 2x - 1 \quad (2.2)$$

Po transformacji obrazów do obiektu typu tensor wartości pikseli zawierają się w przedziale $[0, 1]$. Podstawiając te wartości do wzoru 2.2 otrzymujemy, że po normalizacji wartości pikseli zawierają się w przedziale $[-1, 1]$. Taka normalizacja pozwala na wycentrowanie danych wejściowych wokół zera, co może prowadzić do stabilniejszego przebiegu procesu uczenia oraz efektywniejszej optymalizacji parametrów modelu. Należy również uwzględnić dodatkowy krok transformacyjny w przypadku zbioru MNIST. Ponieważ obrazy w tym zbiorze są jednokanałowe (skala szarości), konieczne było dodanie dodatkowego kroku transformacyjnego, który rozszerza pojedynczy kanał do trzech kanałów, ponieważ taki właśnie format wymagany jest przez architekturę sieci użytej w badaniach.

Metoda `get_dataset()` pozwala na utworzenie odpowiedniego obiektu zbioru danych z biblioteki PyTorch, nakładając jednocześnie zdefiniowane wcześniej kroki transformacyjne. W celu wydzielenia zbioru walidacyjnego służącego do oceny modeli podczas procesu uczenia zaimplementowana została metoda `get_train_val_datasets()`. Omawiana metoda pobiera pełny zbiór treningowy i wydobywa z niego zbiór walidacyjny o wielkości determinowanej przez parametr `val_ratio`, który przyjęto na poziomie 0.1. Metoda `get_test_dataset()` stanowi wygodny sposób na pobranie zbioru testowego dla danego zbioru danych nakładając na niego odpowiednie transformacje. Ostatnią główną funkcjonalnością klasy `DataManager` jest metoda `get_data_loaders()`, która tworzy obiekty `DataLoader` dla zbioru treningowego, walidacyjnego oraz testowego. Obiekty te umożliwiają iterację po danych w trakcie trwania procesu uczenia modeli.

Po przygotowaniu klasy odpowiedzialnej za zarządzanie danymi przystąpiono do implementacji modułu `ModelTrainer`, którego zadaniem było uporządkowanie procesu treningu, walidacji oraz ewaluacji modeli klasyfikacyjnych wykorzystywanych w badaniach. Klasa ta pozwalała oddzielić logikę uczenia modelu od pozostałych elementów projektu.

Obiekt klasy `ModelTrainer` przyjmował model sieci neuronowej, urządzenie wykonujące obliczenia oraz nazwy klas występujących w analizowanym zbiorze danych. W konstruktorze określano również ścieżkę zapisu modelu na podstawie klasy `Paths` oraz tworzono obiekt `TrainingConfig`, z którego pobierano parametry konieczne do przeprowadzenia treningu. Funkcja straty została zdefiniowana jako `CrossEntropyLoss`, natomiast jako optymalizator wykorzystano algorytm Adam. Model był następnie przenoszony na wskazane urządzenie obliczeniowe. Klasa przechowywała również historię procesu uczenia w postaci słownika zawierającego wartości funkcji straty oraz dokładności dla zbioru treningowego i walidacyjnego. Dzięki temu możliwa była późniejsza analiza przebiegu uczenia modelu.

Metoda `train_one_epoch()` realizowała pojedynczą epokę treningową. Dla kolejnych batchy wykonywana była propagacja w przód, obliczanie funkcji straty, propagacja wsteczna oraz aktualizacja wag modelu. Dla każdej epoki obliczana była średnia wartość funkcji straty oraz dokładność klasyfikacji na zbiorze treningowym. W metodzie `validate()` dokonywana była ocena modelu na zbiorze walidacyjnym w danej epoce. Wynikiem działania metody były średnia strata walidacyjna oraz dokładność klasyfikacji na zbiorze walidacyjnym. Metoda `fit()` pełniła rolę orkiestratora dwóch poprzednich metod, wywołując je w pętli po liczbie zadanych epok treningowych. Wyniki metod `train_one_epoch()` i `validate()` zapisywane były w historii treningu. Informacje w niej zawarte, czyli dokładność oraz wartości funkcji straty, były wykorzystywane do oceny jakości procesu uczenia. Po zakończeniu treningu model mógł zostać oceniony na zbiorze testowym za pomocą metody `evaluate()`, która generowała predykcje dla wszystkich przykładów testowych oraz raport klasyfikacyjny zawierający metryki dla poszczególnych klas. Ostatnią funkcjonalnością klasy `ModelTrainer` była metoda `save_model()`, która umożliwiała zapis wag modelu do pliku, dając możliwość załadowania i wykorzystania modelu w późniejszych badaniach.

2.3. Metody generowania perturbacji

W celu generowania zakłóconych wersji obrazów wejściowych wykorzystywanych w eksperymentach nad stabilnością metod XAI zaimplementowano klasę `Perturbations`. Jej zadaniem było przygotowanie danych poddanych perturbacjom, które następnie wykorzystywano do porównania map atrybucji uzyskanych dla obrazów oryginalnych oraz zaburzonych.

Obiekt klasy `Perturbations` przyjmował model predykcyjny oraz urządzenie obliczeniowe. W konstruktorze określono również minimalną i maksymalną dopuszczalną wartość pikseli po perturbacji. Zgodnie z opisem procesu przygotowania danych w sekcji 2.2, dane wejściowe zostały znormalizowane do zakresu $[-1, 1]$, dlatego wartości graniczne ustawiono odpowiednio na -1 oraz 1. Po wygenerowaniu perturbacji wartości obrazu były ograniczane do tego przedziału, co zapewniało zgodność danych wejściowych z zakresem wykorzystywanym podczas trenowania modelu.

Pierwszą zaimplementowaną metodą perturbacji była metoda `gaussian()`, która dodawała do obrazu wejściowego szum Gaussa. W pierwszym kroku tworzono kopię tensora wejściowego i przenoszono ją na wybrane urządzenie obliczeniowe. Następnie generowano tensor losowego szumu o takim samym rozmiarze jak obraz wejściowy. Wygenerowany szum był skalowany przez parametr σ , a następnie dodawany do obrazu zgodnie ze wzorem ???. Wynikowy tensor był ograniczany do zakresu $[-1, 1]$ i zwracany jako zakłócona wersja obrazu wejściowego.

Drugą metodą perturbacji była metoda `fgsm()`, która generowała obraz zaburzony zgodnie z procedurą FGSM opisaną wzorem ??. Metoda przyjmowała tensor wejściowy, klasę docelową oraz wartość parametru ϵ . Na początku tworzono kopię obrazu wejściowego, przenoszono ją na urządzenie obliczeniowe oraz włączano śledzenie gradientów względem wejścia. Następnie obraz przekazywano do modelu, a dla wskazanej klasy docelowej obliczano wartość funkcji straty. Po wyzerowaniu wcześniejszych gradientów wykonywano propagację wsteczną, dzięki czemu uzyskiwano gradient funkcji straty względem obrazu wejściowego. Na jego podstawie tworzono perturbację o skali określonej przez parametr ϵ , którą dodawano do obrazu wejściowego. Po ponownym wyzerowaniu gradientów wynikowy obraz był ograniczany do zakresu $[-1, 1]$ i zwracany jako przykład zaburzony. W obu metodach operacje wykonywane były na kopii tensora wejściowego, tak, aby oryginał został nienaruszony.

2.4. Metody wyjaśniania decyzji modeli

Kolejnym etapem przygotowywania środowiska badawczego było zaimplementowanie klas odpowiedzialnych za generowanie map atrybucji (wyjaśnień): `IGExplainer` oraz `OcclusionExplainer`. Obie klasy wykorzystywały implementacje metod dostępnych w bibliotece `Captum`, a ich zadaniem było ujednolicenie sposobu generowania wyjaśnień oraz przetwarzania otrzymanych atrybucji do formatu wykorzystywanego w dalszych eksperymentach.

Klasa `IGExplainer` została zaimplementowana w celu generowania wyjaśnień metodą `Integrated Gradients`. Do konstruktora klasy przekazywano model predykcyjny oraz urządzenie obliczeniowe. Wewnątrz konstruktora tworzony był obiekt konfiguracyjny `IGConfig`, z którego pobierano parametry metody, takie jak liczba kroków `n_steps`, która stanowiła przybliżenie całkowania ze wzoru ??? oraz wartość bazowa `baseline_value`. Dodatkowo inicjalizowano obiekt `IntegratedGradients` z biblioteki `Captum`, powiązany z analizowanym modelem. Główną funkcjonalność klasy stanowiła metoda `explain()`, która przyjmowała tensor wejściowy oraz identyfikator klasy docelowej. Tensor wejściowy był przenoszony na wskazane urządzenie obliczeniowe, a następnie włączano dla niego śledzenie gradientów. Kolejnym krokiem było utworzenie tensora bazowego o takiej samej strukturze jak obraz wejściowy, wypełnionego wartością określoną w konfiguracji. Atrybucje wyznaczano za pomocą metody

`attribute()`, do której przekazywano obraz wejściowy, tensor bazowy, klasę docelową oraz liczbę kroków określoną w `IGConfig`.

Klasa `OcclusionExplainer` pełniła analogiczną rolę dla metody `Occlusion`. Do jej konstruktora przekazywano model predykcyjny, urządzenie obliczeniowe oraz nazwę analizowanego zbioru danych. Wewnątrz klasy tworzony był obiekt `OcclusionConfig`, zawierający parametry metody osobno dla zbiorów MNIST oraz CIFAR-10. Inicjalizowano również obiekt `Occlusion` z biblioteki `Captum`. W metodzie `explain()` klasy `OcclusionExplainer` tensor wejściowy przenoszono na urządzenie obliczeniowe oraz tworzono tensor bazowy wypełniony wartością `baseline_value`. Następnie, w zależności od wartości pola `dataset`, wybierano odpowiednie parametry `strides` oraz `sliding_window_shapes`. Atrybucje wyznaczano następnie za pomocą metody `attribute()`, przekazując do niej tensor wejściowy, parametry okna, tensor bazowy oraz klasę docelową.

Metoda ta agregowała atrybucje wyznaczone dla poszczególnych kanałów obrazu do jednej mapy atrybucji oraz przygotowywała trzy warianty reprezentacji wyniku: `raw`, `abs` i `deletion`. Pierwszy wariant zachowywał oryginalny znak atrybucji, drugi przedstawiał ogólną siłę wpływu niezależnie od znaku sumując wartości bezwzględne, natomiast trzeci pozostawiał jedynie dodatnie wartości atrybucji. Tak przygotowane mapy były następnie wykorzystywane w metrykach oceny stabilności wyjaśnień modeli.

2.5. Metryki oceny wyjaśnień

W celu mierzalnej oceny stabilności oraz użyteczności map atrybucji zaimplementowano klasę `Metrics`. Klasa ta zawierała zestaw metod obliczających metryki wykorzystywane w dalszych eksperymentach.

Pierwszą zaimplementowaną metryką było podobieństwo cosinusowe, realizowane przez metodę `cosine_similarity()`. Metoda przyjmowała dwie mapy atrybucji, które w pierwszym kroku były przekształcane do postaci jednowymiarowych wektorów. Następnie obliczano stosunek ich iloczynu skalarnego do iloczynu długości obu wektorów zgodnie ze wzorem $\frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$. W celu zabezpieczenia obliczeń przed potencjalnym błędem dzielenia przez zero do mianownika dodawano niewielką stałą $1e-8$.

Drugą zaimplementowaną metryką była Top-K IoU, obliczana przez metodę `topk_iou()`. Podobnie jak w przypadku podobieństwa cosinusowego, mapy atrybucji były najpierw przekształcane do postaci jednowymiarowych wektorów. Następnie na podstawie parametru `top_k_percent` wyznaczano liczbę pikseli należących do grupy najbardziej istotnych elementów. Dla każdej mapy wybierano indeksy pikseli o najwyższych wartościach atrybucji, a wartość metryki wyznaczano jako stosunek liczności części wspólnej tych zbiorów do liczności ich sumy.

Ostatnią zaimplementowaną metryką była Deletion AUC, której obliczanie realizowała metoda `deletion_auc()`. Jej działanie rozpoczynało się od utworzenia kopii obrazu wejściowego oraz uporządkowania pikseli zgodnie z ich istotnością określoną przez przekazaną mapę atrybucji. Następnie, w kolejnych krokach, najbardziej istotne piksele były stopniowo zastępowane wartością bazową. Liczbę etapów tego procesu określał parametr `steps`, natomiast domyślną wartość bazową przyjęto na poziomie `-1`, zgodnie z zakresem wartości danych po normalizacji. Po każdej modyfikacji obrazu wyznaczano prawdopodobieństwo przypisane przez model klasie docelowej i zapisywano je wraz z odpowiadającym mu udziałem usuniętych pikseli. Na podstawie otrzymanych wartości konstruowano krzywą opisującą zmianę pewności modelu wraz ze wzrostem liczby usuniętych pikseli, a następnie obliczano pole pod tą krzywą za pomocą funkcji `auc()` dostępnej w bibliotece `scikit-learn`.

2.6. Orkiestracja eksperymentu

Głównym modulem odpowiedzialnym za przeprowadzanie eksperymentów była klasa `XAIExperiment`. Jej zadaniem była orkiestracja wcześniej przygotowanych elementów środowiska badawczego. Klasa ta umożliwiała przeprowadzenie kompletnej procedury porównywania wyjaśnień uzyskanych dla obrazów oryginalnych oraz ich zaburzonych wersji.

Do konstruktora klasy przekazywano model, urządzenie obliczeniowe, nazwy klas, zestaw poprawnie sklasyfikowanych przykładów oraz jeden z obiektów klas wyjaśniających opisanych w sekcji 2.4. Na podstawie przekazanych przykładów wyznaczano również listę analizowanych klas. Wewnątrz klasy tworzono obiekt `Perturbations`, odpowiedzialny za generowanie zaburzonych obrazów, obiekt `Metrics`, wykorzystywany do obliczania wartości metryk, oraz obiekt `ExperimentConfig`, zawierający parametry eksperymentów.

Metoda `iter_clean_examples()` umożliwiała iterację po wszystkich obrazach wykorzystywanych w eksperymencie. Dla każdej klasy zwracany był identyfikator klasy, numer przykładu oraz tensor obrazu przygotowany w formacie wymaganym przez model. Rozwiązanie to pozwalało wykonywać obliczenia nie tylko dla pojedynczego przykładu danej klasy, lecz także dla wielu przykładów, co umożliwiało późniejsze uśrednianie wyników. Do generowania zaburzonych wersji obrazów wykorzystano metodę `get_perturbed_tensor()`. W zależności od wartości parametru `perturbation_type` metoda wywoływała odpowiednią funkcję klasy `Perturbations`. Dla wartości `gaussian` generowany był obraz z dodanym szumem Gaussa, natomiast dla wartości `fgsm` tworzony był obraz zaburzony metodą FGSM. Metoda `run_similarity_metrics()` odpowiadała za obliczanie metryk podobieństwa pomiędzy mapami atrybucji obrazów oryginalnych i zaburzonych. W pierwszym kroku tworzono pamięć podręczną map atrybucji dla obrazów oryginalnych, aby uniknąć ich wielokrotnego przeliczania dla każdej wartości perturbacji. Do obliczania podobieństwa wykorzystywano wariant mapy `abs`, czyli sumę wartości bezwzględnych atrybucji po kanałach obrazu. Wybór tego wariantu wynikał z celu analizowanych metryk. Podobieństwo cosinusowe

oraz Top-K IoU miały oceniać przede wszystkim stabilność położenia obszarów istotnych dla modelu, a nie kierunek ich wpływu na predykcję. Z tego względu uwzględniano całkowitą siłę atrybucji niezależnie od jej znaku. Następnie dla kolejnych wartości parametru perturbacji generowano zaburzone obrazy, obliczano dla nich mapy atrybucji oraz porównywano je z wcześniej zapisanymi mapami obrazów oryginalnych. Wyniki zapisywano w postaci listy słowników zawierających typ perturbacji, wartość parametru, identyfikator klasy, nazwę klasy, numer przykładu oraz wartości obliczonych metryk. Ostatecznie wyniki zwracano jako obiekt `DataFrame` biblioteki `pandas`. Metoda `run_deletion_auc()` została przygotowana do oceny zmian wartości metryki Deletion AUC dla obrazów oryginalnych i zaburzonych. W tym przypadku wykorzystywano wariant mapy `deletion`, zawierający jedynie dodatnie wartości atrybucji. Taki wybór był związany z charakterem procedury Deletion AUC, w której kolejno zastępowano wartością bazową piksele uznane za najbardziej wspierające predykcję analizowanej klasy. Ujemne wartości atrybucji mogły oznaczać obszary działające przeciwnie względem danej klasy, dlatego nie były uwzględniane podczas wyboru pikseli usuwanych w tej metryce. Podobnie jak w przypadku metryk podobieństwa, najpierw tworzono pamięć podręczną wyników dla obrazów oryginalnych. Dla każdego przykładu zapisywano predykcję modelu, pewność modelu względem klasy docelowej oraz wartość Deletion AUC obliczoną na podstawie mapy `deletion`. W dalszej części procedury, dla każdej wartości perturbacji, generowano obraz zaburzony i sprawdzano predykcję modelu oraz jego pewność względem klasy docelowej. Następnie wyznaczano mapę atrybucji dla obrazu zaburzonego i obliczano wartość Deletion AUC.

W klasie zaimplementowano również metody pomocnicze służące do agregacji wyników. Metoda `make_pivot()` tworzyła tabelę przestawną, w której wartości wybranej metryki były uśredniane dla poszczególnych klas i poziomów perturbacji. Metoda `summarize_similarity()` grupowała wyniki metryk podobieństwa względem wartości perturbacji i obliczała średnie oraz odchylenia standardowe dla podobieństwa cosinusowego i Top-K IoU. Analogicznie metoda `summarize_deletion()` agregowała wyniki związane z Deletion AUC, pewnością modelu oraz liczbą zmian predykcji.

Zastosowanie klasy `XAIExperiment` pozwoliło uporządkować przebieg eksperymentów i zapewnić jednolity sposób obliczania wyników dla różnych metod wyjaśniania, typów perturbacji oraz analizowanych zbiorów danych.

3. Trening modeli oraz przebieg badań

W niniejszym rozdziale przedstawiono przebieg procesu trenowania modeli predykcyjnych użytych w badaniach. Opisano również konfigurację oraz przebieg całych eksperymentów dla rozważanych zbiorów danych.

3.1. Trening modeli klasyfikacyjnych

Przygotowanie badań rozpoczęto od wytrenowania modelu klasyfikacyjnego dla zbioru MNIST. Zdecydowano się na Wykorzystanie architektury ResNet18 z wagami wstępnie wytrenowanymi, a ostatnią warstwę klasyfikacyjną zastąpiono nową warstwą w pełni połączoną, dostosowaną do liczby klas występujących w danym zbiorze. W przypadku zbiorów MNIST oraz CIFAR-10 liczba klas wynosiła 10.

Proces dostrajania zrealizowano z wykorzystaniem wcześniej opisanej w sekcji 2.2 klasy `ModelTrainer`. Dla obu zbiorów zastosowano ten sam schemat treningu, obejmujący uczenie na zbiorze treningowym oraz kontrolę jakości na zbiorze walidacyjnym dla ostatniej epoki treningowej. Dodatkowo w tabeli zamieszczono również ocenę na zbiorze testowym. Różnice pomiędzy eksperymentami dotyczyły wykorzystywanego zbioru danych oraz liczby epok dostrajania modelu (dla zbioru MNIST liczba epok równa 3, dla CIFAR-10 liczba epok wynosiła 7). Większa liczba epok dla CIFAR-10 wynikała z większej złożoności danych. Dla każdego zbioru monitorowano wartość funkcji straty oraz dokładność klasyfikacji na zbiorze treningowym i walidacyjnym. Po zakończeniu dostrajania modele oceniono na zbiorach testowych. Wartości metryk dla rozważanych zbiorów przedstawiono w tabeli 3.1.

Tabela 3.1. Zestawienie wartości dokładności dla zbioru MNIST oraz CIFAR-10 dla wszystkich etapów treningu

Zbiór danych	Dokładność treningowa	Dokładność walidacji	Dokładność testowa
MNIST	0.9915	0.9938	0.9937
CIFAR-10	0.9707	0.8862	0.8806

Uzyskane wyniki wskazują, że oba modele osiągnęły skuteczność wystarczającą do dalszych badań nad stabilnością metod wyjaśniania. W przypadku zbioru MNIST dokładność na zbiorach treningowym, walidacyjnym i testowym była bardzo zbliżona oraz przekraczała 99%. Dla zbioru CIFAR-10 uzyskano niższą dokładność niż dla MNIST, co było oczekiwane ze względu na większą złożoność obrazów naturalnych oraz większą różnorodność cech wizualnych. Widoczna różnica pomiędzy dokładnością treningową a walidacyjną i testową wskazuje, że zadanie klasyfikacji było trudniejsze, a model w większym stopniu dopasował

się do danych treningowych. Mimo to dokładność testowa na poziomie 88% została uznana za wystarczającą do przeprowadzenia dalszych eksperymentów.

3.2. Konfiguracja i przebieg eksperymentów

4. tutaj badamy wyniki mnista

5. tutaj badamy wyniki cifara

6. tutaj podsumowujemy co sie wydarzylo